SOLACE
CARE

Running Solace locally

How to start the infrastructure, the four backend services, the admin portal and the mobile app on one computer, and how to sign in and test once they run.

For	Developers and testers setting up Solace Care on their own computer.
Covers	Installing the tools, starting each part in order, signing in as a staff member, seeker or specialist, and the problems people hit on Windows.
Built from	The repo's READMEs (root, infra, admin, mobile, Keycloak), docker-compose and each service's application.yml, checked against the code on 22 October 2026.

Contents

How to read this guide . . . 3
1. Install the tools . . . 4
2. Start the infrastructure . . . 5
3. Run the four backend services . . . 6
4. Run the admin portal . . . 7
5. Run the mobile app . . . 8
6. Sign in and test . . . 9
7. Stop everything . . . 10

How to read this guide

Start the parts in this order: install the tools once, start the infrastructure in Docker, run the four backend services, then the admin portal and the mobile app. Each section has the commands to copy.

Commands are written for Git Bash on Windows (or any Unix shell). PowerShell differences are noted where they matter. Run backend commands from **backend/** and everything else from the repo root unless the step says otherwise.

Local only. Every password and key here is a development default. Real environments take secrets from Vault.

Part	Address	Notes
Gateway (the API the apps call)	http://localhost:8080/api	Routes /core, /tracking and /ai
Core	REST :8101, gRPC :9101	
Tracking	REST :8102, gRPC :9102	
AI	REST :8103, gRPC :9103	
Admin portal	http://localhost:4200	
Keycloak	http://localhost:8180	Console login admin / admin
PostgreSQL 16	localhost:5432	Databases core, tracking, ai, keycloak
Kafka	localhost:9092	Schema registry on :8081
Redis	localhost:6379	
MinIO	API :9000, console :9001	Login solace / solace-dev-only
LiveKit	ws://localhost:7880	Key devkey
Grafana	http://localhost:3000	Login admin / admin

STEP 1

Install the tools

Install these once on the computer, then install the JavaScript packages and build the shared ones.

JDK	21. Gradle finds a local JDK 21 or downloads one.
Node.js	22 or later
pnpm	10, through corepack
Docker	With Compose v2

Step by step

1 **Turn on pnpm**

```
corepack enable
```

2 **Install and build the JavaScript workspace**

From the repo root. Installing also generates the design tokens and the staff sign-in theme; building makes the shared packages (api-clients, i18n, tokens).

```
pnpm install
pnpm build
```

STEP 2

Start the infrastructure

Docker runs everything the services need: PostgreSQL with pgvector, Kafka and its schema registry, Redis, MinIO, Keycloak with both realms imported, LiveKit, and Grafana with Prometheus, Loki and Tempo.

Step by step

1. **Build the Keycloak sign-in extension**
   Needed before Keycloak's first start: it provides the one-time code sign-in and the temporary password rules.

   ```
   cd backend && ./gradlew :keycloak:otp-authenticator:jar && cd ..
   ```

2. **Start the stack**
   The first run pulls the images.

   ```
   docker compose -f infra/docker-compose.yml up -d
   ```

3. **Check it**
   Waits for every component and checks it answers.

   ```
   bash infra/scripts/smoke-check.sh
   ```

4. **Optional: audio recording for the AI scribe**
   Starts LiveKit Egress, which records consented sessions into a private bucket.

   ```
   docker compose -f infra/docker-compose.yml --profile scribe up -d
   ```

Rules behind this flow

- To change a local password, copy **infra/.env.example** to **infra/.env** and edit it.
- Topics are not created automatically; each service declares its own when it starts.

STEP 3

Run the four backend services

Open one terminal for each service, inside backend/. Core needs three settings that choose local stand-ins for payments, SMS and email, and push notifications; without the first two it refuses to start.

Step by step

1 **Core**

SOLACE_PAYMENTS_PROVIDER=sandbox uses the sandbox payment provider, so no real money moves. SOLACE_NOTIFICATIONS_SENDER=logging writes SMS and emails to the log. The push setting is optional; without it Core uses Expo's push service.

```
SOLACE_PAYMENTS_PROVIDER=sandbox \
SOLACE_NOTIFICATIONS_SENDER=logging \
SOLACE_NOTIFICATIONS_PUSH_SENDER=logging \
  ./gradlew :services:core:bootRun
```

2 **In PowerShell instead**

Set the variables first, then run the same command.

```
$env:SOLACE_PAYMENTS_PROVIDER='sandbox'
$env:SOLACE_NOTIFICATIONS_SENDER='logging'
$env:SOLACE_NOTIFICATIONS_PUSH_SENDER='logging'
./gradlew :services:core:bootRun
```

3 **Tracking**

```
./gradlew :services:tracking:bootRun
```

4 **AI**

```
./gradlew :services:ai:bootRun
```

5 **Gateway**

The only address the apps call: http://localhost:8080/api.

```
./gradlew :services:gateway:bootRun
```

6 **Optional: check the whole path**

From the repo root, with all four running. Signs a test seeker in, calls a ping through the gateway, and checks that one trace in Grafana covers the gateway, Core, the gRPC calls and the Kafka event.

```
bash infra/scripts/e2e-ping.sh
```

If something goes wrong

- **Core stops at start with "solace.payments.provider must be notchpay or sandbox"** or a missing sender: the variables above are not set in that terminal.
- **Build fails with "class file major version 69"**: Gradle ran on JDK 25 (VS Code's Gradle does this). Run ./gradlew from a terminal with JDK 21.

STEP 4

Run the admin portal

The portal runs on http://localhost:4200 and signs staff in through Keycloak. The first staff account is made by hand in the Keycloak console.

Step by step

1. **Start it**
   From the repo root.
   ```
   pnpm --filter @solace/admin start
   ```
2. **Open the Keycloak console**
   http://localhost:8180, login admin / admin, realm solace-staff.
3. **Create a staff user**
   Add a user with an email, set a temporary password, and give the role SUPER_ADMIN.
4. **Sign in at http://localhost:4200**
   With the email and the temporary password. The first sign-in asks for a new password (at least 12 characters), then an authenticator app: scan the QR code with any TOTP app.
5. **Create more staff from the portal**
   Team and roles. Approving a specialist and approving refunds need a second person, so make a second account to test those.

Rules behind this flow

- The later-phase screens (tracking modules, health check, AI controls, copilot) are hidden locally by **admin/src/environments/environment.ts**, where **flags** are all false. Set them to true to see those screens.

STEP 5

Run the mobile app

One Expo app for guests, seekers and specialists. It uses native modules, so it runs as a development build, not in Expo Go.

Step by step

1 **Build and install the dev client**

On an Android device or emulator, from the repo root.

```
pnpm --filter @solace/mobile android
```

2 **Start Metro**

For a dev client that is already installed.

```
pnpm --filter @solace/mobile start
```

3 **Point the app at your computer**

On a phone or emulator, localhost is the phone itself. The Android emulator reaches your computer as 10.0.2.2; a real phone needs your computer's LAN address.

```
EXPO_PUBLIC_API_URL=http://10.0.2.2:8080/api \
EXPO_PUBLIC_KEYCLOAK_URL=http://10.0.2.2:8180 \
  pnpm --filter @solace/mobile start
```

4 **Quick preview without a device**

Renders the screens in a browser. Good for layout, copy and colour only.

```
cd mobile && npx expo start --web
```

If something goes wrong

- **Android build fails on Windows** with "Filename longer than 260 characters" or "build.ninja still dirty": turn on Windows long paths (an administrator sets LongPathsEnabled = 1) and use a newer Android SDK CMake, or build on EAS, macOS or Linux (mobile/README.md).

STEP 6

Sign in and test

Nothing is sent to real phones or wallets locally. Codes and messages go to the logs, and payments go through the sandbox.

Step by step

1. **Seeker and specialist sign-in codes**
   Keycloak writes each code to its log.

   ```
   docker compose -f infra/docker-compose.yml logs keycloak | grep "sign-in code"
   ```

2. **Guest booking codes and other SMS**
   They appear in Core's log, because the sender is set to logging.

3. **Paying**
   The sandbox provider approves a payment about 5 seconds after it starts.

4. **A specialist**
   Sign up as a specialist in the app, then review and approve the application in the admin portal with two different staff accounts.

5. **Later-phase features**
   Tracking, the health check, AI assist, the scribe, the companion, the copilot and phone dial-in are off. Switch them on in the admin portal under Settings and flags.

6. **AI features**
   They need a model in solace.ai.models. With none, each one uses its fallback: the clinicians' wording, the brief without an AI summary, reviews waiting for a person, an empty note to write, the companion's fixed questions.

STEP 7

Stop everything

Stop the services with Ctrl+C in their terminals, then stop Docker.

Step by step

1 **Stop and keep the data**

```
docker compose -f infra/docker-compose.yml down
```

2 **Stop and delete all data**

Use this for a clean start: databases, Keycloak users and MinIO files all go.

```
docker compose -f infra/docker-compose.yml down -v
```